

РОССИЙСКИЙ УНИВЕРСИТЕТ ДРУЖБЫ НАРОДОВ

Факультет физико-математических и естественных наук

Кафедра: Бизнес Информатика

Архитектура Компьютеров и операционные системы

Лабораторная работа №4 : Продвинутое использование git

МВЕТУВОП ФОНГАНГ ДЖУЛС ФЕРРИ

1132255560

Группа: НБИбд-02-25

Москва, 2026

Лабораторная работа №4: Продвинутое использование git

Цель работы

Получение навыков правильной работы с репозиториями git, включая организацию ветвления, управление версиями и стандартизацию коммитов.

Выполнение работы

1. Установка программного обеспечения

1.1 Установка git-flow

Первым делом я установил git-flow на свою систему Fedora. Для этого я выполнил следующие команды:

text

```
dnf copr enable elegos/gitflow
dnf install gitflow
```

После установки я проверил, что git-flow успешно установлен, выполнив команду `git flow version`.

1.2 Установка Node.js и pnpm

Для работы с инструментами семантического версионирования я установил Node.js и pnpm:

text

```
dnf install nodejs
dnf install pnpm
```

1.3 Настройка Node.js

Я выполнил настройку pnpm для добавления исполняемых файлов в переменную PATH:

text

```
pnpm setup
source ~/.bashrc
```

1.4 Установка commitizen и standard-changelog

Для форматирования коммитов и генерации журналов изменений я установил необходимые пакеты:

text

```
pnpm add -g commitizen
pnpm add -g standard-changelog
```

2. Создание и настройка репозитория

2.1 Создание репозитория на GitHub

Я создал новый репозиторий на GitHub с названием `git-extended`. После этого я инициализировал локальный репозиторий и выполнил первый коммит:

text

```
git init
git add .
git commit -m "first commit"
git remote add origin git@github.com:julesfongang/git-extended.git
git push -u origin master
```

2.2 Инициализация package.json

Я выполнил инициализацию пакета Node.js:

text

```
pnpm init
```

В процессе заполнил следующие параметры:

- Название пакета: `git-extended`
- Версия: `1.0.0`
- Описание: `Git repo for educational purposes`
- Точка входа: `index.js`
- Репозиторий git: `git@github.com:julesfongang/git-extended.git`
- Автор: `Jules Ferry MVETUVOP FONGANG <julesfongang@github.com>`
- Лицензия: `CC-BY-4.0`

2.3 Настройка commitizen

Я отредактировал файл `package.json`, добавив конфигурацию для commitizen:

```
json
"config": {
  "commitizen": {
    "path": "cz-conventional-changelog"
  }
}
```

После редактирования файл `package.json` приобрёл следующий вид:

```
json
{
  "name": "git-extended",
  "version": "1.0.0",
  "description": "Git repo for educational purposes",
  "main": "index.js",
  "repository": "git@github.com:julesfongang/git-extended.git",
  "author": "Jules Ferry MVETUVOP FONGANG <julesfongang@github.com>",
  "license": "CC-BY-4.0",
  "config": {
    "commitizen": {
      "path": "cz-conventional-changelog"
    }
  }
}
```

2.4 Первый коммит с commitizen

Я добавил все файлы в индекс и выполнил коммит с использованием commitizen:

```
text
git add .
git cz
```

В интерактивном меню я выбрал:

- Тип коммита: `feat` (новая функция)
- Описание: `initial commit with package.json`
- Дополнительные опции оставил пустыми

После этого я отправил изменения на GitHub:

text

```
git push
```

3. Инициализация git-flow

3.1 Запуск инициализации

Я выполнил инициализацию git-flow:

text

```
git flow init
```

В процессе инициализации я ответил на вопросы следующим образом:

- Production branch: `master` (оставил по умолчанию)
- Development branch: `develop` (оставил по умолчанию)
- Feature branch prefix: `feature/` (оставил по умолчанию)
- Release branch prefix: `release/` (оставил по умолчанию)
- Hotfix branch prefix: `hotfix/` (оставил по умолчанию)
- Support branch prefix: `support/` (оставил по умолчанию)
- Version tag prefix: установил `v`

3.2 Проверка текущей ветки

Я проверил, на какой ветке нахожусь:

text

```
git branch
```

Ветка `develop` была активна, что соответствует ожиданиям.

3.3 Отправка всех веток на GitHub

Я отправил все ветки в удалённый репозиторий:

text

```
git push --all
```

3.4 Настройка вышестоящей ветки

Я установил внешнюю ветку как вышестоящую для ветки `develop`:

text

```
git branch --set-upstream-to=origin/develop develop
```

4. Создание первого релиза

4.1 Создание релизной ветки

Я создал релизную ветку для версии 1.0.0:

text

```
git flow release start 1.0.0
```

4.2 Генерация журнала изменений

Я сгенерировал журнал изменений для первого релиза:

text

```
standard-changelog --first-release
```

Эта команда создала файл `CHANGELOG.md` с описанием изменений.

4.3 Добавление журнала изменений в коммит

Я добавил файл `CHANGELOG.md` в индекс и создал коммит:

text

```
git add CHANGELOG.md
git commit -am 'chore(site): add changelog'
```

4.4 Завершение релиза

Я завершил работу над релизом:

text

```
git flow release finish 1.0.0
```

В процессе выполнения команды я оставил предложенные сообщения для коммитов слияния и тега по умолчанию.

4.5 Отправка изменений на GitHub

Я отправил все изменения и теги на GitHub:

text

```
git push --all
git push --tags
```

4.6 Создание релиза на GitHub

Я создал релиз на GitHub с использованием утилиты `gh`:

text

```
gh release create v1.0.0 -F CHANGELOG.md
```

5. Разработка новой функциональности

5.1 Создание функциональной ветки

Я создал ветку для новой функциональности:

text

```
git flow feature start feature_branch
```

5.2 Добавление нового файла

Я создал новый файл для реализации функциональности:

text

```
touch feature.js  
echo "// New feature: user authentication" > feature.js
```

5.3 Коммит изменений

Я добавил файл в индекс и создал коммит:

text

```
git add feature.js  
git commit -m "feat: add user authentication feature"
```

5.4 Завершение работы над функциональностью

Я завершил работу над функциональностью и объединил ветку с develop:

text

```
git flow feature finish feature_branch
```

6. Создание нового релиза с версией 1.2.3

6.1 Создание релизной ветки

Я создал новую релизную ветку для версии 1.2.3:

text

```
git flow release start 1.2.3
```

6.2 Обновление версии в package.json

Я открыл файл package.json и изменил версию на 1.2.3.

6.3 Обновление журнала изменений

Я сгенерировал обновлённый журнал изменений:

text

```
standard-changelog
```

6.4 Коммит изменений

Я добавил обновлённый журнал изменений в индекс и создал коммит:

text

```
git add CHANGELOG.md
git commit -am 'chore(site): update changelog for version 1.2.3'
```

6.5 Завершение релиза

Я завершил работу над релизом:

text

```
git flow release finish 1.2.3
```

6.6 Отправка изменений на GitHub

Я отправил все изменения и теги на GitHub:

text

```
git push --all
git push --tags
```

6.7 Создание релиза на GitHub

Я создал релиз на GitHub с описанием из журнала изменений:

text

```
gh release create v1.2.3 -F CHANGELOG.md
```

7. Работа с ветками исправления (hotfix)

7.1 Обнаружение критической ошибки

В процессе тестирования версии 1.2.3 была обнаружена критическая ошибка в файле `feature.js`.

7.2 Создание ветки hotfix

Я создал ветку для срочного исправления:

text

```
git flow hotfix start hotfix_branch
```

7.3 Исправление ошибки

Я отредактировал файл `feature.js`, исправив критическую ошибку:

text

```
echo "// Fixed critical bug: authentication bypass" > feature.js
```

7.4 Коммит исправления

Я добавил изменения в индекс и создал коммит:

text

```
git add feature.js
git commit -m "fix: resolve critical authentication bypass bug"
```

7.5 Завершение работы над hotfix

Я завершил работу над исправлением:

text

```
git flow hotfix finish hotfix_branch
```

7.6 Отправка изменений на GitHub

Я отправил все изменения и теги на GitHub:

text

```
git push --all
git push --tags
```

8. Просмотр истории и результатов

8.1 Просмотр истории коммитов

Я просмотрел историю коммитов в графическом виде:

text

```
git log --oneline --graph --all --decorate
```

8.2 Просмотр журнала изменений

Я проверил содержимое файла CHANGELOG.md:

text

```
cat CHANGELOG.md
```

8.3 Проверка на GitHub

Я открыл репозиторий на GitHub и проверил:

- Наличие всех веток (master, develop)
 - Наличие созданных релизов (v1.0.0, v1.2.3)
 - Корректность отображения журнала изменений в релизах
-

Выводы

В ходе выполнения лабораторной работы мною были получены практические навыки правильной работы с репозиториями git. Я выполнил следующие задачи:

Установка и настройка программного обеспечения:

- Установил git-flow на систему Fedora.
- Установил Node.js и npm.
- Установил и настроил commitizen для интерактивного создания коммитов.
- Установил standard-changelog для генерации журналов изменений.

Создание и конфигурация репозитория:

- Создал репозиторий на GitHub с названием git-extended.
- Инициализировал локальный репозиторий и выполнил первый коммит.
- Настроил package.json для работы с conventional commits.
- Выполнил коммит с использованием commitizen.

Настройка git-flow:

- Инициализировал git-flow с установкой префикса тегов в v.
- Создал и настроил ветки master и develop.
- Отправил все ветки на GitHub.

Создание релизов:

- Создал релизную ветку для версии 1.0.0.
- Сгенерировал журнал изменений с помощью standard-changelog.
- Завершил релиз и создал тег v1.0.0.
- Опубликовал релиз на GitHub.

Разработка новой функциональности:

- Создал функциональную ветку feature_branch.
- Добавил новый файл feature.js.
- Выполнил коммит с типом feat.
- Завершил работу над функциональностью и объединил с develop.

Создание нового релиза:

- Создал релизную ветку для версии 1.2.3.
- Обновил версию в package.json.
- Сгенерировал обновлённый журнал изменений.
- Завершил релиз и создал тег v1.2.3.

- Опубликовал релиз на GitHub.

Работа с hotfix:

- Создал ветку hotfix для срочного исправления ошибки.
- Внёс исправления в файл feature.js.
- Выполнил коммит с типом fix.
- Завершил работу над hotfix и объединил с master и develop.

Проверка результатов:

- Просмотрел историю коммитов в графическом виде.
- Проверил содержимое CHANGELOG.md.
- Убедился, что все релизы корректно отображаются на GitHub.

Таким образом, цель лабораторной работы полностью достигнута. Я освоил продвинутые техники работы с git, включая:

- Модель ветвления Gitflow с использованием постоянных веток master и develop, а также временных веток feature, release и hotfix.
- Семантическое версионирование с правильным увеличением MAJOR, MINOR и PATCH версий.
- Спецификацию Conventional Commits с правильным оформлением типов коммитов.
- Использование инструментов автоматизации: commitizen для создания коммитов и standard-changelog для генерации журналов изменений.
- Интеграцию с GitHub через создание релизов и тегов.

Приобретённые навыки являются основой для профессиональной разработки программного обеспечения в команде и будут активно применяться в дальнейшей учебной и профессиональной деятельности.